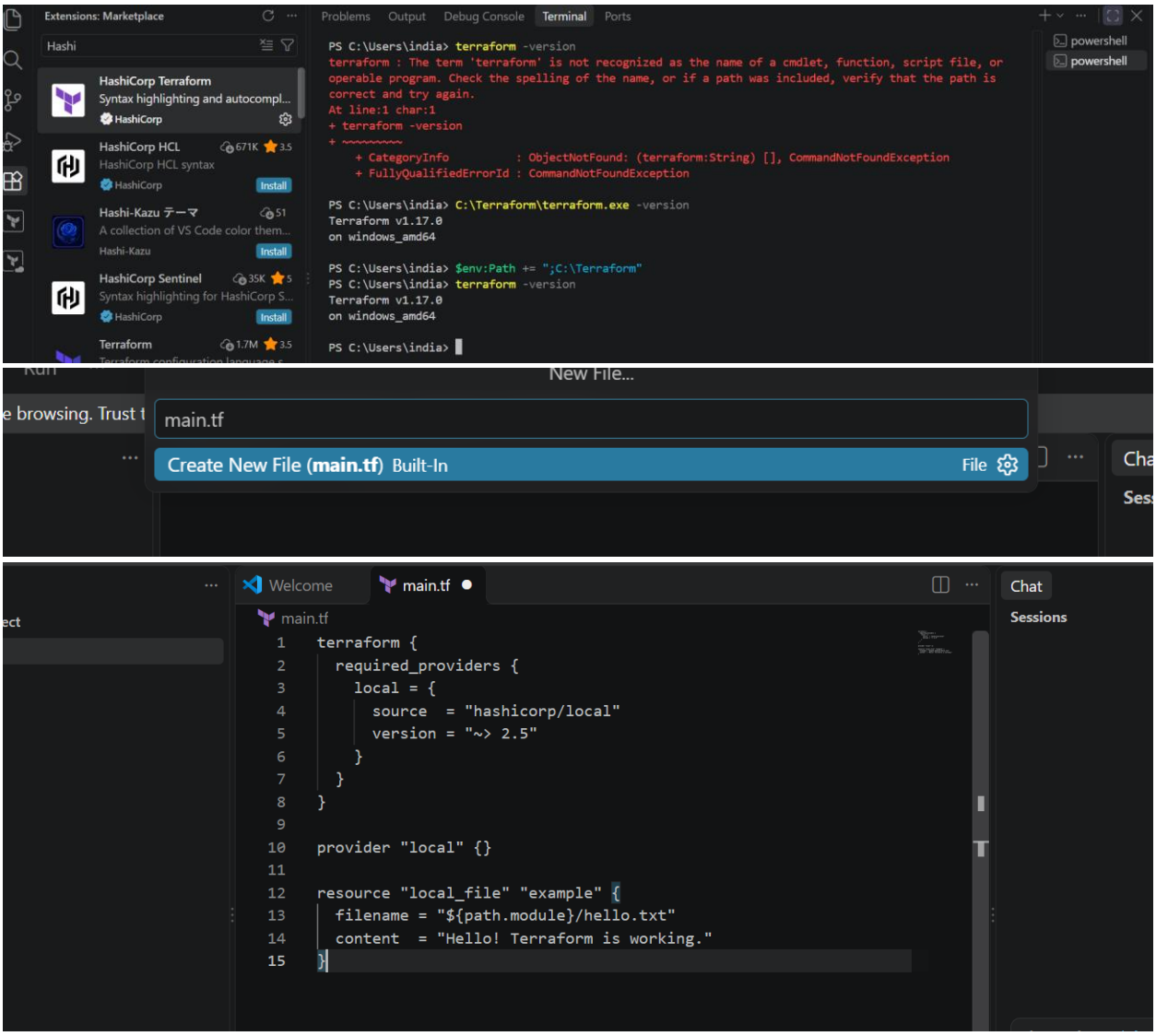
Terraform-1 Task

1). Install Terraform on your PC

2). Execute all the templates shown in video.

3). Note down below points, `Terraform Init` `Terraform Plan` `Terraform Apply` `Terraform Provider`.

4). Integrate a sample Terraform template in Jenkins.



```
PS C:\Terraform-Project> terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/local versions matching "~> 2.5"...
- Installing hashicorp/local v2.9.1...
- Installed hashicorp/local v2.9.1 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Terraform-Project>
```

```
PS C:\Terraform-Project> terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are
indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # local_file.example will be created
  + resource "local_file" "example" {
      + content              = "Hello! Terraform is working."
      + content_base64sha256 = (known after apply)
      + content_base64sha512 = (known after apply)
      + content_md5          = (known after apply)
      + content_sha1         = (known after apply)
      + content_sha256       = (known after apply)
      + content_sha512       = (known after apply)
      + directory_permission = "0777"
      + file_permission      = "0777"
      + filename             = "./hello.txt"
      + id                   = (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.
```

```
PS C:\Terraform-Project> terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are
indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # local_file.example will be created
  + resource "local_file" "example" {
      + content              = "Hello! Terraform is working."
      + content_base64sha256 = (known after apply)
      + content_base64sha512 = (known after apply)
      + content_md5          = (known after apply)
      + content_sha1         = (known after apply)
      + content_sha256       = (known after apply)
      + content_sha512       = (known after apply)
      + directory_permission = "0777"
      + file_permission      = "0777"
      + filename             = "./hello.txt"
      + id                   = (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.
```

```
Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

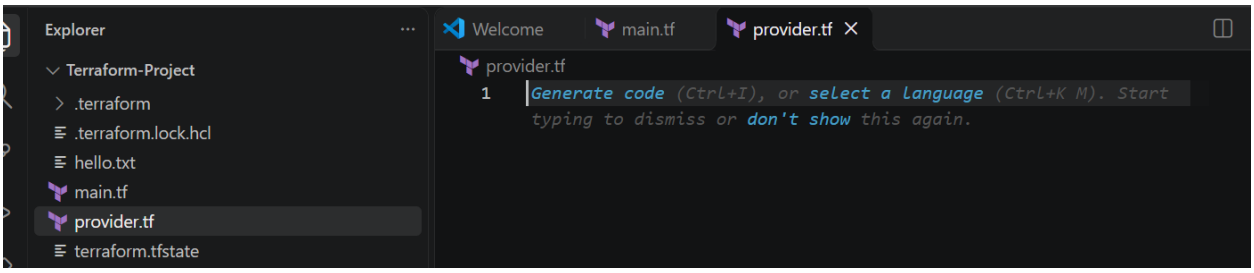
  Enter a value: yes

local_file.example: Creating...
local_file.example: Creation complete after 0s [id=b9baf6c85748e1c8784bd0231804299b7fa1536d]

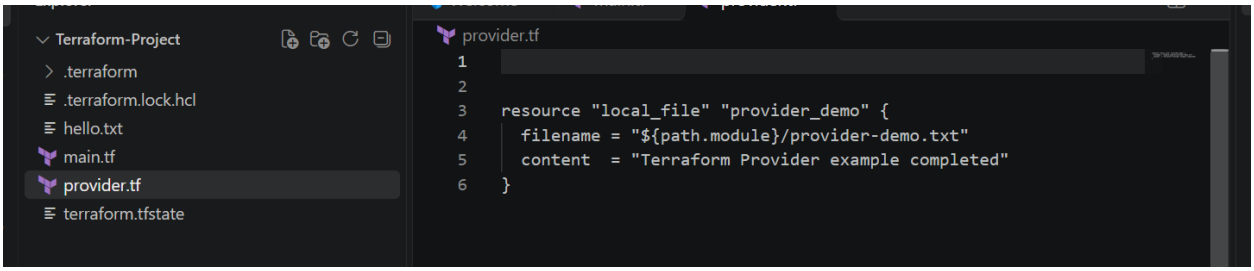
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\Terraform-Project>
```

```
PS C:\Terraform-Project> Get-Content .\hello.txt
Hello! Terraform is working.
PS C:\Terraform-Project>
```

Create provider.tf file



Then save the file



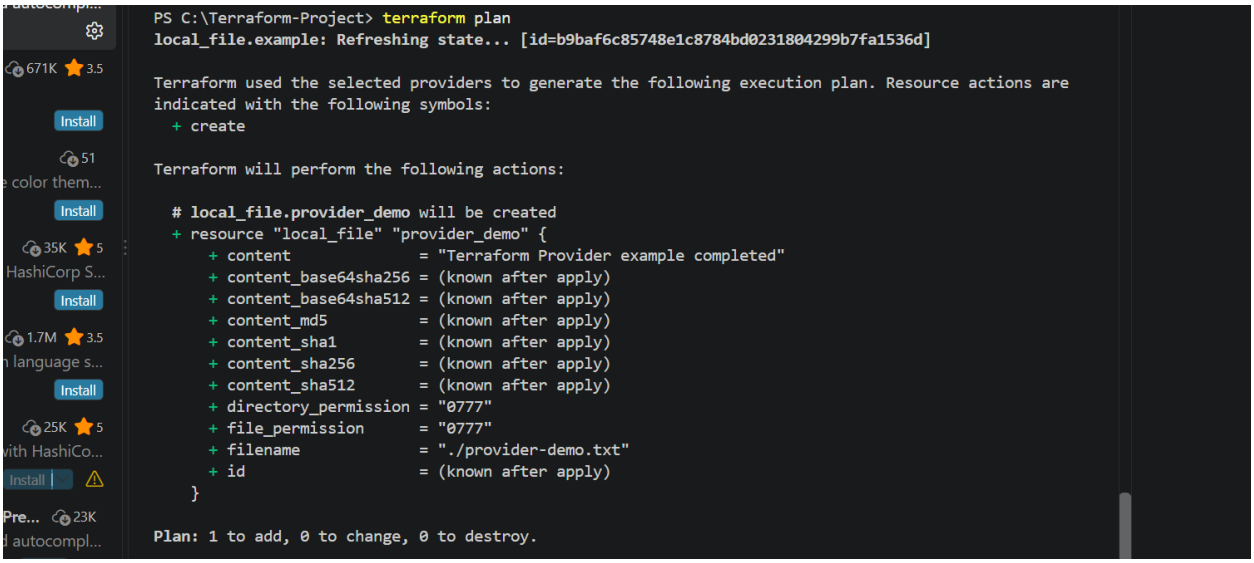

```
PS C:\Terraform-Project> terraform init
Initializing the backend...

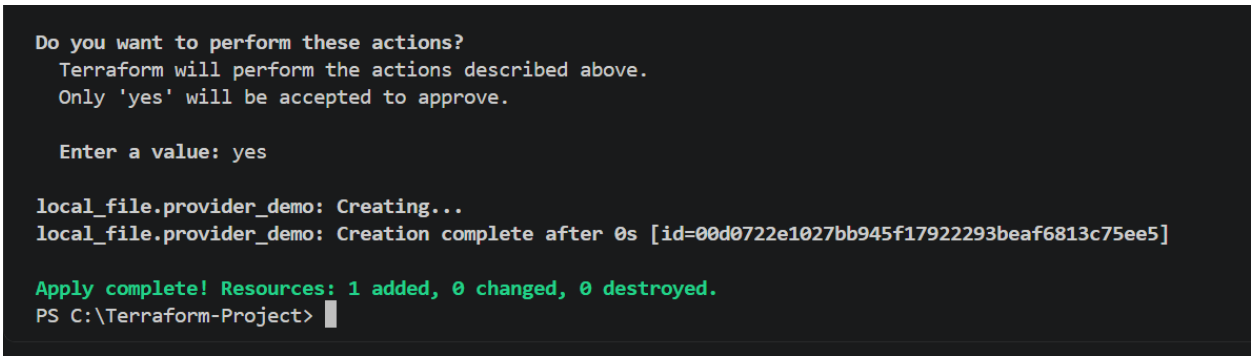
Initializing provider plugins...
- Reusing version 2.9.1 of hashicorp/local from the dependency lock file
- Using previously-installed hashicorp/local v2.9.1

Terraform has been successfully initialized!

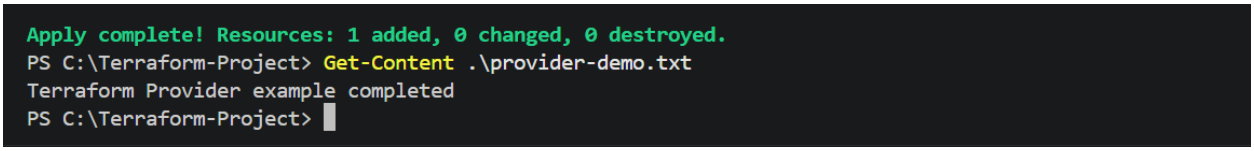
You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

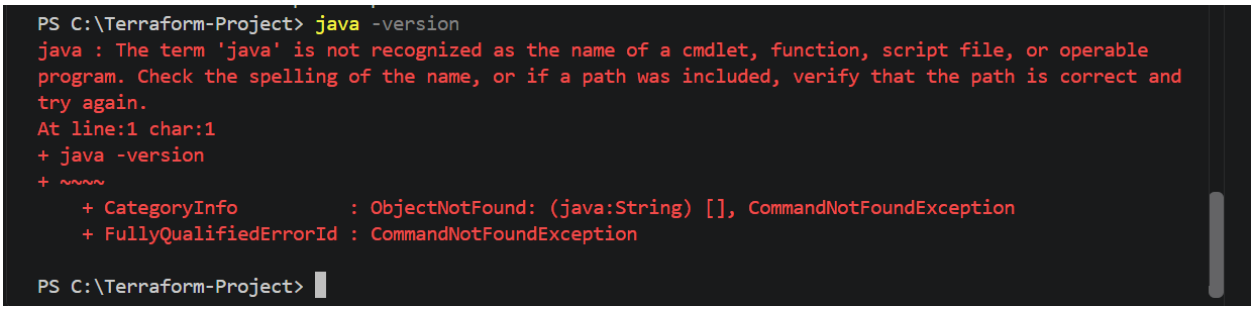
If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

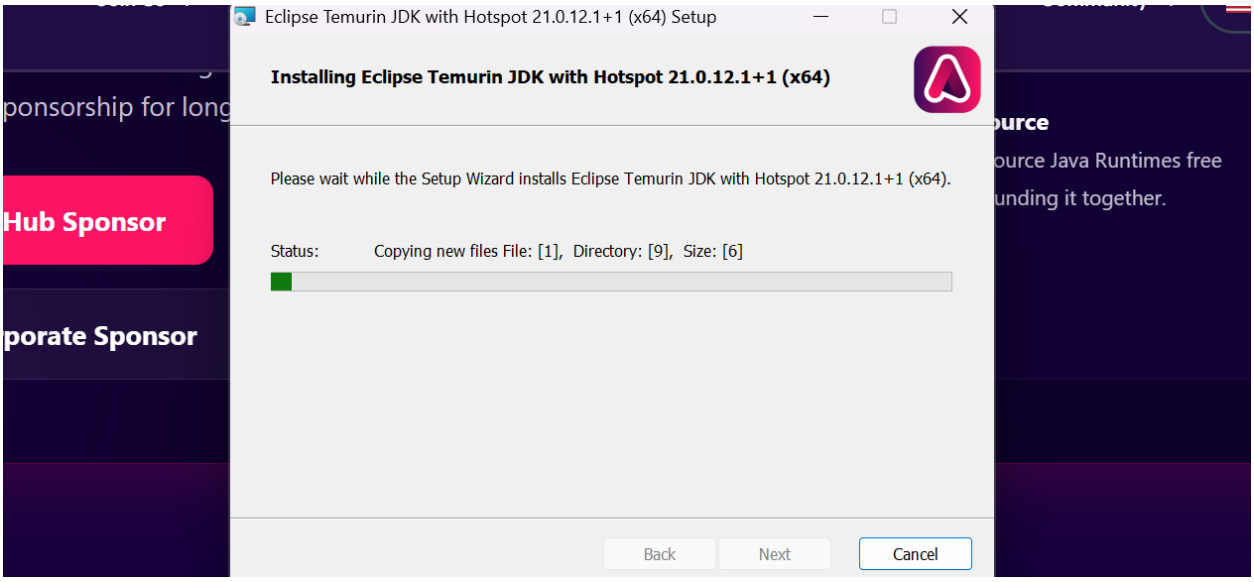




terraform apply is done successfully








```
PS C:\Terraform-Project> java -version
openjdk version "21.0.12.1" 2026-08-18 LTS
OpenJDK Runtime Environment Temurin-21.0.12.1+1 (build 21.0.12.1+1-LTS)
OpenJDK 64-Bit Server VM Temurin-21.0.12.1+1 (build 21.0.12.1+1-LTS, mixed mode, sharing)
PS C:\Terraform-Project>
```

create a jenkins instance

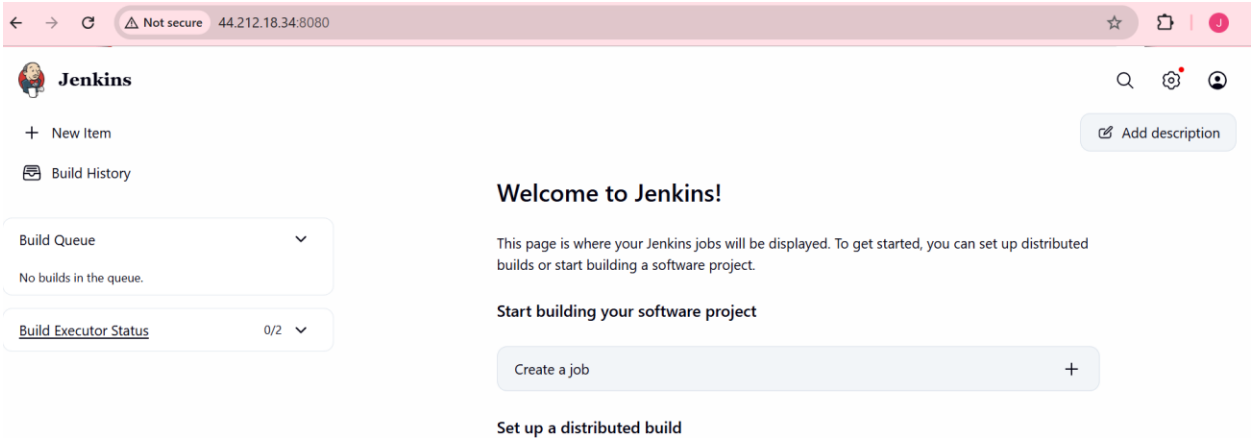
```
Complete!
[root@ip-10-0-2-47 ~]# java --version
openjdk 21.0.13 2026-10-20 LTS
OpenJDK Runtime Environment Corretto-21.0.13.0.10 (build 21.0.13+10-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.13.0.10 (build 21.0.13+10-LTS, mixed mode, sharing)
[root@ip-10-0-2-47 ~]#
```

Install java

```
[root@ip-10-0-2-47 ~]# dnf install jenkins -y
Jenkins-stable                                                  18 kB/s | 833  B     00:00
Jenkins-stable                                                 580 kB/s | 1.6 kB     00:00
Importing GPG key 0x14ABFC68:
 Userid     : "Jenkins Project <jenkinsci-board@googlegroups.com>"
 Fingerprint: 5E38 6EAD B55F 0150 4CAE 8BCF 7198 F4B7 14AB FC68
 From       : https://pkg.jenkins.io/rpm-stable/repodata/repomd.xml.key
Jenkins-stable                                                  74 kB/s | 3.0 kB     00:00
Dependencies resolved.
================================================================================
 Package                Architecture            Version                Repository                Siz
```

Install the dependencies for jenkins and install jenkins

```
[root@ip-10-0-2-47 ~]# systemctl enable jenkins
Created symlink /etc/systemd/system/multi-user.target.wants/jenkins.service → /usr/lib/systemd/system/jenkins.service.
[root@ip-10-0-2-47 ~]# systemctl start jenkins
^[[A[root@ip-10-0-2-47 ~]# systemctl start jenkins^C
[root@ip-10-0-2-47 ~]# systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
     Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
     Active: active (running) since Tue 2026-10-27 16:26:16 UTC; 9s ago
```



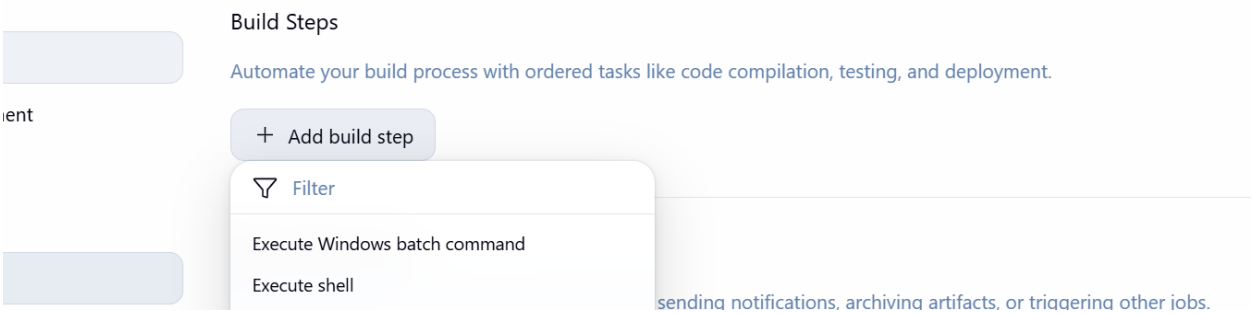
Now install terraform on the jenkins ec2

```
[root@ip-10-0-2-47 ~]# dnf install -y dnf-plugins-core
Last metadata expiration check: 0:07:27 ago on Tue Oct 27 16:24:37 2026.
Package dnf-plugins-core-4.3.0-13.amzn2023.0.7.noarch is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-10-0-2-47 ~]# dnf config-manager --add-repo https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
Adding repo from: https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
[root@ip-10-0-2-47 ~]# dnf install -y terraform
Hashicorp Stable - x86_64                                                    64 MB/s | 1.9 MB     00:00
Last metadata expiration check: 0:00:01 ago on Tue Oct 27 16:32:37 2026.
Dependencies resolved.
================================================================================
 Package                 Architecture            Version                 Repository                Size
================================================================================
Installing:
 terraform               x86_64                  1.17.0-1                hashicorp                 37 M
Installing dependencies:
 git                     x86_64                  2.50.1-1.amzn2023.0.1   amazonlinux               53 k
```

```
[root@ip-10-0-2-47 ~]# terraform -version
Terraform v1.17.0
on linux_amd64

[root@ip-10-0-2-47 ~]# which terraform
/usr/bin/terraform
[root@ip-10-0-2-47 ~]#
```





In the build steps choose execute shell

#!/bin/bash

set -e

terraform -version

cd /home/ec2-user

terraform init

terraform plan

```
[root@ip-10-0-2-47 ~]# mkdir -p /home/ec2-user/terraform-project
[root@ip-10-0-2-47 ~]# chown -R ec2-user:ec2-user /home/ec2-user/terraform-project
[root@ip-10-0-2-47 ~]# nano /home/ec2-user/terraform-project/main.tf
[root@ip-10-0-2-47 ~]#
```

```
  GNU nano 8.3                                  /home/ec2-user/terraform-project/main.tf
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

provider "local" {}

resource "local_file" "example" {
  filename = "${path.module}/hello.txt"
  content  = "Hello! Terraform is working."
}
```

```
[root@ip-10-0-2-47 ~]# nano /home/ec2-user/terraform-project/provider.tf
[root@ip-10-0-2-47 ~]# ls -l /home/ec2-user/terraform-project/
total 8
-rw-r--r--. 1 root root 262 Oct 27 16:39 main.tf
-rw-r--r--. 1 root root 142 Oct 27 16:40 provider.tf
[root@ip-10-0-2-47 ~]#
```

create main.tf and provider.tf

```
Started by user admin
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/Terraform-Job
[Terraform-Job] $ /bin/bash /tmp/jenkins6921023087991343648.sh
/tmp/jenkins6921023087991343648.sh: line 4: cd: /home/ec2-user/terraform-project: Permission denied
Build step 'Execute shell' marked build as failure
Finished: FAILURE
```

Failure due to permissions

```
[root@ip-10-0-2-47 ~]# chmod 755 /home/ec2-user
chmod -R 755 /home/ec2-user/terraform-project
[root@ip-10-0-2-47 ~]# ls -ld /home/ec2-user /home/ec2-user/terraform-project
drwxr-xr-x. 4 ec2-user ec2-user 99 Oct 27 16:37 /home/ec2-user
drwxr-xr-x. 2 ec2-user ec2-user 40 Oct 27 16:40 /home/ec2-user/terraform-project
[root@ip-10-0-2-47 ~]#
```

```
[1mPlan:[0m [0m2 to add, 0 to change, 0 to destroy.
[0m[1mlocal_file.example: Creating...[0m[0m
[0m[1mlocal_file.provider_demo: Creating...[0m[0m
[0m[1mlocal_file.provider_demo: Creation complete after 0s
[id=00d0722e1027bb945f17922293beaf6813c75ee5][0m
[0m[1mlocal_file.example: Creation complete after 0s [id=b9baf6c85748e1c8784bd0231804299b7fa1536d][0m
[0m[1m[32m
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.[0m
Finished: SUCCESS
```

build success!